

Final Report: Autonomous CI/CD Build Assistant

Saim Jawed
saim.jawed@accenture.com
Accenture AEH Internship

June 29, 2026

Abstract

This report outlines the design, development, and deployment of an agentic AI assistant capable of automating CI/CD build failure recovery. Over a 4-week sprint, I transformed manual log debugging into an autonomous process by building a composite GitHub Action powered by Google Gemini. This document details what was built, the system architecture, major technical roadblocks overcome, and the future roadmap.

1 What Was Built (The MVP)

The AI Build Assistant is a production-ready, composite GitHub Action that acts as an autonomous agent within the CI/CD pipeline. When a build fails, the assistant intercepts the logs, diagnoses the root cause using a Large Language Model (Google Gemini), and takes autonomous action to resolve it.

1.1 Key Features Delivered

- **Intelligent Diagnostics:** Maps raw, unstructured build logs to over 10 distinct failure categories (e.g., `dependency_error`, `network_timeout`, `oom_error`, `test_failure`).
- **Actionable PR Comments:** Generates a human-readable diagnostic report directly on the failing Pull Request, including confidence levels, root cause analysis, and step-by-step fix instructions.
- **One-Click Code Fixes:** Emits exact file diffs. Developers can comment `/apply-fix` on the PR, and the assistant will automatically commit the suggested changes to their branch.

- **Deep Scanning:** For massive structural errors, users can comment `/deep-scan` to bypass token-efficient snippets and force the AI to read the entire file context.
- **Autonomous Retries for Transient Errors:** If an infrastructure error is detected (e.g., a network timeout), the assistant autonomously triggers a pipeline rerun via a `retry.yml` workflow, completely removing human intervention for flaky builds.

2 System Architecture & The Agentic Loop

The system is built on the **Observe** → **Reason** → **Act** agentic framework.

2.1 Phase 1: Observe (Context Gathering)

When `ci.yml` fails, `action.yml` is triggered. The Python CLI (`assistant.py`) parses the `build_failure.log`. It then scans the workspace for files modified in the PR. Using a token-efficient pre-scan strategy, it extracts the relevant structural context from the codebase without hitting context-window limits.

2.2 Phase 2: Reason (LLM Analysis)

The log and file contexts are sent to Gemini via a strict structured JSON schema prompt. Gemini reasons over the logs to produce a `FailureDiagnosis` object containing:

- Failure category and Root cause string
- Fix steps array
- Suggested code changes (Filepath, Search string, Replace string)

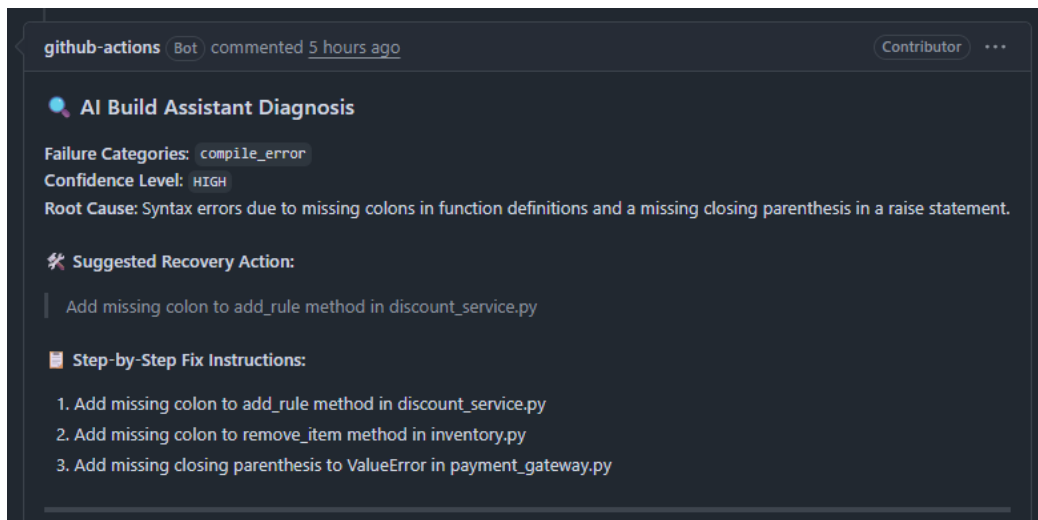


Figure 1: Example of the AI Assistant’s diagnostic report and generated PR comment.

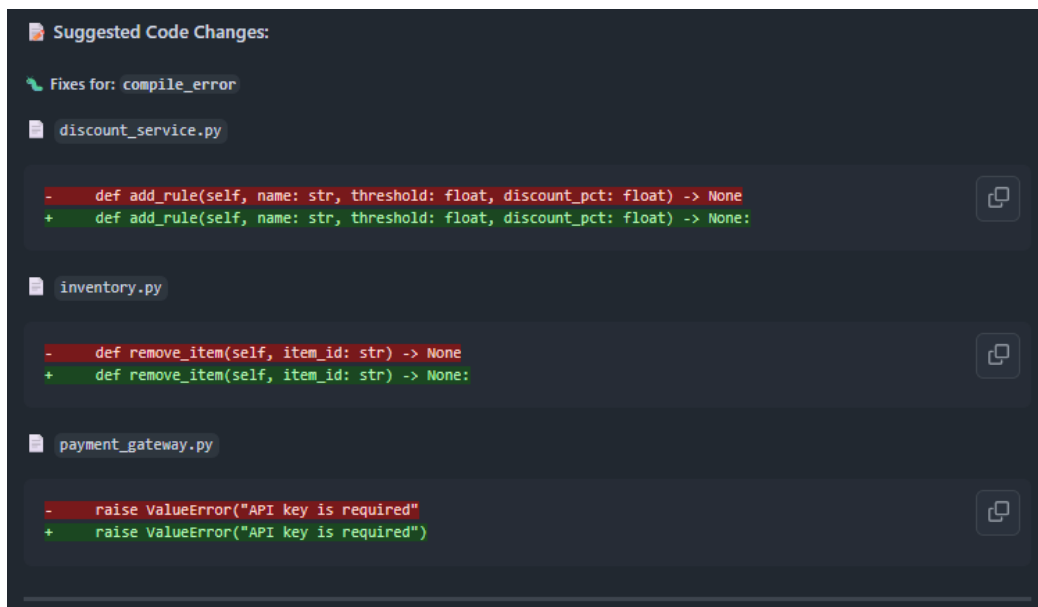


Figure 2: Example of an autonomous code fix applied by the agent.

2.3 Phase 3: Act (Execution)

The `agent.py` orchestrator takes the JSON and executes actions:

1. **Transient Handling:** Flags transient errors and uploads a `transient-status` artifact. A secondary workflow listens for this and executes a GitHub CLI `rerun` command.
2. **PR Commenting:** Interacts with the GitHub REST API to post a Markdown-formatted diagnostic report on the PR.
3. **Memory Storage:** To prevent infinite retry loops or duplicate suggestions, it embeds hidden HTML metadata in the PR comments to maintain state across stateless GitHub Action runner teardowns.

3 Edge-Cases and Safety Guardrails

Building AI agents requires strict guardrails to prevent runaway compute costs and destructive actions.

- **Infinite Loop Prevention:** The autonomous retry workflow is hardcapped at 3 attempts (`run_attempt < 3`). Furthermore, `agent.py` checks its embedded PR comment metadata history to refuse to suggest a fix that has already been tried and failed.
- **Graceful Degradation:** If the Gemini API is unresponsive or rate-limited, the system handles the exception gracefully without crashing the pipeline.

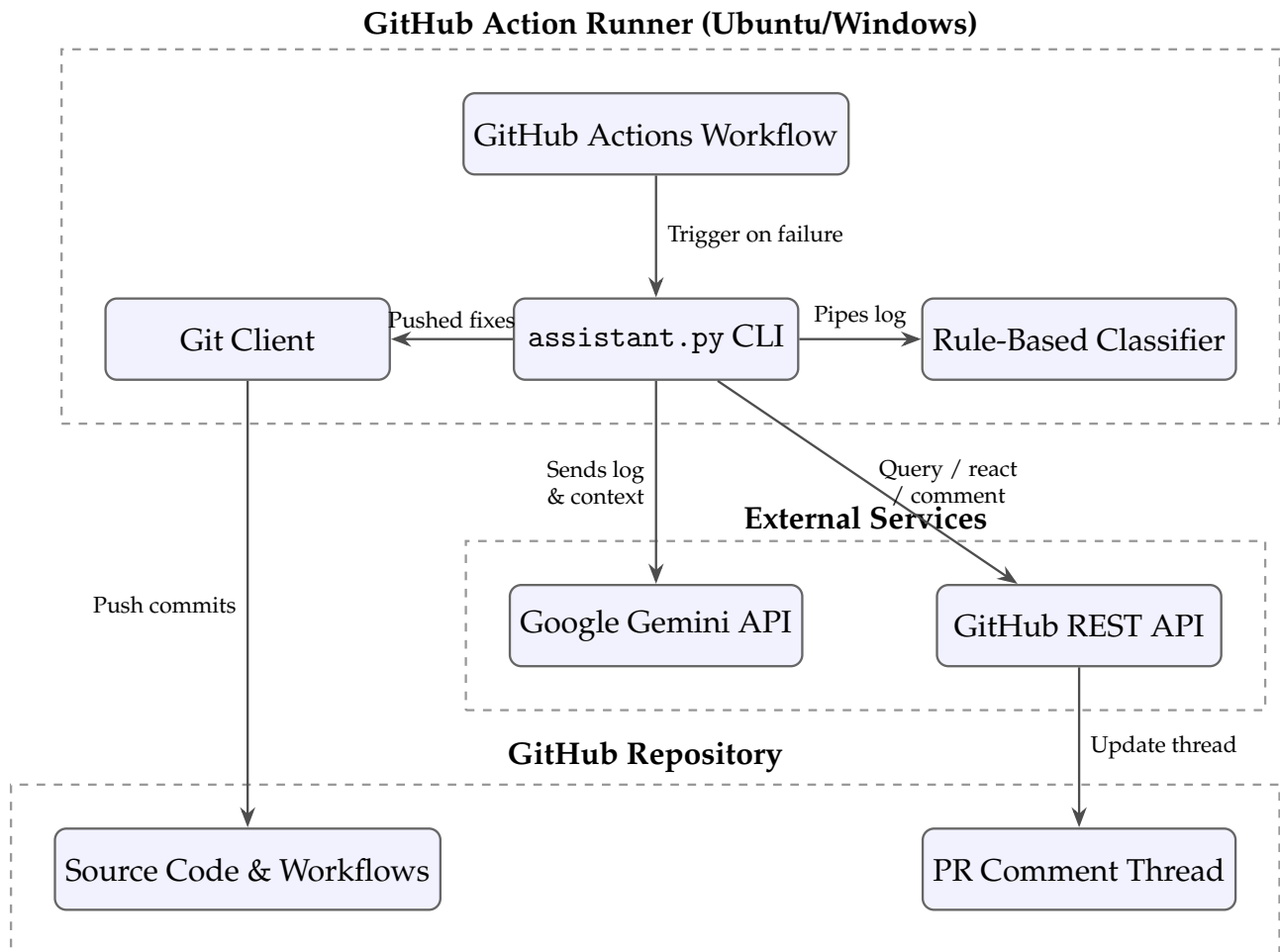


Figure 3: System Architecture & Data Flow Diagram

- **Prompt Injection Defense:** File patches are strictly verified against the actual workspace file contents before being applied via the `/apply-fix` command.

4 Project Observations & Technical Road-blocks

Throughout the 4-week sprint, I documented and overcame several significant technical challenges:

- **State Persistence on Stateless Runners:** GitHub Actions spins up a fresh virtual machine for every run, wiping local JSON memory files. I engineered a stateless memory system by embedding hidden HTML metadata directly into the PR comments, allowing the bot to reconstruct its history by parsing the PR timeline.
- **The `/apply-fix` vs `/deep-scan` Collision:** The bot's own comments contained instructions on how to use `/deep-scan`. I had

to refactor the scanning logic to use strict `startswith()` checks on stripped comment bodies to differentiate user commands from the bot's own output.

- **Context Strategy (Snippets vs. Full Code):** I implemented a two-tier "multi-layer scanning" strategy to balance token exhaustion and API costs against diagnostic accuracy.
- **API Timeouts on Large Contexts:** Deep scanning heavily broken codebases caused the Gemini API to timeout at 30 seconds. I increased the global timeout to 120 seconds to accommodate large-scale reasoning.
- **Multi-File Structural Errors (The Ultimate Test):** I designed an intricate `test-multi-file-error` branch in a testing repository, introducing missing dependencies across multiple files while breaking function signatures, forcing the AI to holistically correct multiple source files at once without relying on test assertions.

5 Next Steps & Future Roadmap

With the MVP finalized, future iterations of the product could explore:

1. **Multi-Model Support:** Decouple the `llm_client.py` to allow easy swapping between Google Gemini, OpenAI GPT-4o, and Anthropic Claude 3.5 Sonnet based on user preference or token cost.
2. **Vector-Search Context:** Implement a lightweight embeddings database (e.g., ChromaDB) to retrieve project-wide context, allowing the assistant to fix deep architectural regressions rather than just localized syntax errors.
3. **Web Dashboard:** Create a standalone React dashboard to track metrics (e.g., "Hours saved by Auto-Retry", "Most common failure categories") across an entire GitHub organization.